

CMSC 471 — Introduction to Artificial Intelligence

Programming Assignment 1: A* Search and Heuristics with DOGCAT

Instructor: Dr. KMA Solaiman

Total: 100 points

Due: See the course site

Learning objectives

After completing this assignment, you should be able to:

- formulate a search problem using the AIMA Problem interface;
- implement state transitions, goal testing, and cumulative path costs;
- design and justify admissible heuristics for different cost functions;
- compare uniform-cost search and A* empirically;
- distinguish action reversibility from cost symmetry; and
- critically verify an AI-generated heuristic.

1. The DOGCAT game

DOGCAT is a word-transformation game. You are given two words with the same number of letters. Your goal is to transform the initial word into the goal word by replacing one letter at a time.

Every intermediate result must be a legal word. For this assignment, a word is considered legal if it appears in the provided words34.txt.gz dictionary.

For example:

dog → cog → cot → cat

This is legal because each action changes exactly one letter and every resulting word is in the provided dictionary. The following is not a legal one-step transformation:

cat → tac

because two letter positions change.

For this assignment:

- states are lowercase three- or four-letter words;
- the initial and goal words must have the same length;
- both must appear in the provided dictionary; and
- every action preserves the length of the word.

2. Cost measures

Your implementation must support three ways of measuring path cost.

2.1 Steps

Every legal letter replacement costs 1. Therefore, the total cost is the number of actions in the solution.

For example:

dog → cog → cot → cat

has a steps cost of 3.

2.2 Scrabble cost

The cost of an action is determined by the value of the replacement letter:

Replacement letters Cost
A, E, I, O, U, L, N, S, T, R 1
D, G 2
B, C, M, P 3
F, H, V, W, Y 4
K 5
J, X 6
Q, Z 10

For example:

dog → cog → cot → cat

has a Scrabble cost of:

$$3 + 1 + 1 = 5$$

because the newly inserted letters are c, t, and a.

Use the values specified above, even if another Scrabble reference uses different values.

2.3 Frequency cost

The dictionary associates each word with a nonnegative rarity value.

When an action produces word $\diamond\diamond$, its cost is:

$$1 + \text{rarity}(\diamond\diamond)$$

The initial word does not contribute a frequency cost. Each resulting word does.

3. Starter code and environment

Your repository contains:

- dc.py, the file you must complete;
- dcsolve.py, which solves and displays individual problems;
- dctest.py, which runs several test problems;
- words34.txt.gz, the supplied dictionary and rarity data;
- search.py and utils.py, the supplied AIMA search files; and
- any additional supporting files included in the repository.

Use Python 3.

Use the supplied versions of search.py, utils.py, and the other starter files. Keep them in the same directory as dc.py. Do not replace them with files downloaded from another version of the AIMA repository.

4. Programming requirements [60 points]

Complete the DC class in dc.py. It must extend the supplied AIMA Problem class and solve any valid three- or four-letter DOGCAT instance.

Complete the following methods:

__init__

Initialize the problem using:

- the initial word; goal
- the goal word; and

- one of the cost measures: steps, scrabble, or frequency.

Validate the supplied arguments. The initial and goal words must:

- be lowercase strings;
- contain either three or four letters;
- have the same length; and
- appear in the provided dictionary.

The cost argument must be one of the three supported values.

actions

Return all legal one-letter replacement actions from the given state.

A legal action must:

- change exactly one position;
- replace the existing character with a different character; and
- produce a word contained in the provided dictionary.

Use the action representation specified in the starter code.

result

Return the state produced by applying a legal action to the current state.

goal_test

Return True exactly when the given state equals the goal state.

path_cost

Return the cumulative cost of reaching the new state.

Its behavior must depend on the selected cost measure:

- steps: add 1;
- scrabble: add the assigned value of the newly inserted letter; or
- frequency: add $1 + \text{rarity}(\text{word})$, where word is the resulting word.

h

Return a nontrivial admissible estimate of the remaining cost.

Your heuristic must:

- depend on the selected cost measure;
- return 0 at the goal;
- never overestimate the true minimum remaining cost; and
- not be the trivial heuristic that always returns 0.

You will define and justify each heuristic in the written portion.

__repr__

Return a clear string representation of the problem instance, including the initial word, goal word, and cost measure.

You may write auxiliary functions when useful.

Test your code using both `dcsolve.py` and `dctest.py`. Example commands include:

```
python dcsolve.py dog ca
python dcsolve.py why ask steps
python dcsolve.py bear duck frequency
python dctest.py
```

```
python dcsolve.py dog cat
```

```
dc(dog,cat,steps) cost: 3.00; time:0.001; solution: dog cog cot cat; deltas:[0.0, 0.0, 0.0]
```

```
python dcsolve.py why ask
```

```
dc(why,ask,steps) cost: 5.00; time:0.004; solution: why wha aha ahs ass ask; deltas:[-2, -1, -1, 0.0, 0.0]
```

```
python dcsolve.py test bush
```

```
dc(test,bush,steps) cost: 3.00; time:0.001; solution: test best bust bush; deltas:[0.0, 0.0, 0.0]
```

```
python dcsolve.py love hate
```

```
dc(love,hate,steps) cost: 3.00; time:0.001; solution: love hove have hate; deltas:[0.0, 0.0, 0.0]
```

```
python dcsolve.py bear duck steps
```

```
dc(bear,duck,steps) cost: 4.00; time:0.002; solution: bear beak beck buck duck; deltas:[0.0, 0.0, 0.0, 0.0]
```

```
python dcsolve.py bear duck scrabble
```

```
dc(bear,duck,scrabble) cost: 11.00; time:0.002; solution: bear beak beck buck duck; deltas:[0.0, 0.0, 0.0, 0.0]
```

```
python dcsolve.py bear duck frequency
```

```
dc(bear,duck,frequency) cost: 38.24; time:1.025; solution: bear beak beck buck duck; deltas:[-27.011, -15.546, -7.034, 0.0]
```

Your exact runtime may differ. Your program may also return a different optimal path, and your deltas may differ depending on the admissible heuristic you implement.

- Under the steps objective, an optimal alternative must have the same number of steps.
- Under the Scrabble and frequency objectives, an optimal alternative must have the same total cost, but it may contain a different number of steps.

Interpreting deltas

A positive delta proves that the heuristic overestimated at that node. Zero or negative deltas are expected for an admissible heuristic, but observing them only along one returned path does not prove that the heuristic is admissible for every state.

Measuring nodes expanded

For the experiments in Written Question 2, use the reporting behavior already available in the supplied search functions:

```
uniform_cost_search(problem, display=True)
astar_search(problem, display=True)
```

Run every search using a new DC problem instance.

For this assignment, **nodes expanded** means the number reported by the supplied search function when `display=True`. Do not use runtime as the primary measure of heuristic quality because runtime can vary across machines and executions.

6. Written analysis [40 points]

Submit your answers in a PDF named `pa01.pdf`.

Support your conclusions using your implementation, formal reasoning, and experimental results. A returned path may differ from the sample output while still being correct if it has the same optimal cost.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—**steps**, **Scrabble**, and **frequency**—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining cost it lower-bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

Cost type “steps”

1. If `state == goal` return 0

Heuristic cost = 0

For `i` in `range(len(state))`

If `node.state[i] != goal[i]`

`Heuristic_cost ++`

Return `heuristic_cost`

2. Every mismatch in a word will need an action and for every action the cost goes up by 1
3. Each action needed costs exactly 1. So whether the path is optimal or not, the heuristic cost will always be less than or equal to the path so there cannot be an overestimate.

Cost type “scrabble”

1. If `state == goal` return 0

Heuristic cost = 0

For `i` in `range(len(state))`

If `node.state[i] != goal[i]`

`Heuristic_cost += scrabble_values[goal[i]]`

Return `heuristic_cost`

2. Every mismatch in a word will need an action and for every action the cost goes up by the scrabble cost for the specific letter
3. The heuristic accurately represents the cost of adding the correct letter for every mismatch and will never go above the intended cost for adding that specific letter and so every path will cost at least the estimated heuristic cost

Cost type “frequency”

1. If state == goal return 0
Heuristic cost = 0
For i in range len(state)
 If node.state[i] != goal[i]
 Heuristic_cost ++
Heuristic_cost += dictionary[goal]
Return heuristic_cost
2. Every mismatch in a word will need an action and with every action the cost goes up by 1 and then the frequency cost of the final step is also added to the heuristic at the end of the loop
3. The heuristic adds 1 for the amount of steps needed to reach the goal and then it adds the frequency cost for the last step. But since the heuristic doesn't consider the frequency cost of the in between steps it will never overestimate the actual cost

Run both **uniform-cost search** and **A*** on:

- WHY → ASK
- BEAR → DUCK

Run each problem using all three cost measures. Report:

Word pair Cost measure UCS cost UCS nodes expanded A* cost A* nodes expanded Reduction

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.3%
WHY → ASK	Frequency	38.85	808	38.85	634	21.53%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.8%
BEAR → DUCK	Frequency	38.24	1084	38.24	302	72.14%

Calculate the percentage reduction as:

$$100 \times \frac{\text{UCS expansions} - \text{A* expansions}}{\text{UCS expansions}}$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy
Yes for all 6 tests UCS and A* returned the same optimal cost
2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?
For scrabble A* produced the largest percentage reduction in expanded nodes, at 99.8% for bear->duck and 98.3% for why->ask
3. For each word pair, under which cost measure was the heuristic least informative?

The heuristic was least informative for frequency with a 21.5% reduction for why->ask and a 72.1% reduction for bear->duck

4. Explain the differences using the definitions of your heuristics, not runtime alone.

The difference exists because some of the heuristics provide a closer estimate to the real cost than others. The steps cost is pretty effective since it counts mismatched positions creating a lower bound of the number of actions left. Scrabble is the closest estimate to the real cost because it provides the cost of every letter replacement, so A* wouldn't make very costly moves unless absolutely necessary. And the frequency heuristic is the worst because it doesn't consider the frequency cost of every new word created, it only considers the frequency cost of the last word and the mismatched letters. Not considering the frequency cost of all the in-between words leaves out a lot of the real cost, making it the least accurate heuristic resulting in a lower reduction rate.

Different optimal paths are acceptable when their total costs are equal.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—**steps**, **Scrabble**, and **frequency**—determine whether the optimal cost from w_1 to w_2 must always equal the optimal cost from w_2 to w_1 .

For each cost measure:

1. State **yes** or **no**.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the provided

dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

For steps the optimal cost would be the same because the only thing that adds to the cost is the number of actions and the number of actions would be the same.

For scrabble the answer is no because the cost is determined based on the types of letters used to replace a letter. For example while going from dog to cog (dog->cog) switching the d to c costs 3. But while reversing the action cog->dog switching the c to d costs 2. Switching different letters have different costs and so the optimal cost will not always be the same if reversed.

For frequency the answer is also no because the cost depends on the cost of its frequency in the dictionary and different words have different frequencies. For example the word dog has a frequency of 5.819 and cog has 11.393. So going from dog to cog will cost $5.8 + 1$ for action = 6.8 while going from cog to dog will cost $11.39 + 1$ for action = 12.39 . So the optimal cost for the reverse will not be the same as the original optimal cost.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(w)$ is the number of positions in which state w differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic h_0

(n) is the number of positions in which state n differs from the goal. Propose

a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0

for every

state and strictly larger for at least one state, without overestimating the true optimal remaining cost.



The Proposed Heuristic

Let g be the target goal word. For any word state n of length L , let $\text{Diff}(n, g) \subseteq \{1, \dots, L\}$ be the set of index positions where n and g differ.

Define $c_{\min}$ as the global minimum cost of entering any word in the dictionary:

$$c_{\min} = 1 + \min_{w \in \text{Dict}} \text{rarity}(w)$$

Because $\text{rarity}(w) \geq 0$, we know $c_{\min} \geq 1$.

We define the rarity-aware heuristic $h_1(n)$ as:

$$h_1(n) = h_0(n) \cdot c_{\min}$$

where $h_0(n) = |\text{Diff}(n, g)|$ is the standard Hamming distance (number of mismatched character positions).

2. A precise formula or pseudocode for the proposed heuristic.

min_rarity = minimum rarity of any word in the dictionary

Min_cost_of_entry = 1 + min_rarity

Mismatched_spots = 0

For each letter in state:

 If state[letter] != goal[letter]:

 Mismatched_spots ++;

Return mismatched_spots * min_cost_of_entry

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

Let $k = h_0(n)$

Where k is the number of mismatches in letters between the current word and the goal. Since every action changes 1 letter, every path from state n to goal will need at least k number of actions

Let $c = 1 + \text{minimum rarity of any word in dictionary}(w)$

Every action will result in a word in the dictionary and costs 1 for the step + the rarity of the word. Therefore every action costs at least c

Therefore every path to the goal is at least $k * c$

$h_1(n) = k * c = h_0(n) * c$

Therefore $h_1(n) \leq$ the optimal cost to the goal and $h_1(n)$ never overestimate and is admissible

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

The proposal was valid

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2.

Compare

solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

For why->ask my heuristic h_0 expanded 755 nodes while the ai's proposed heuristic expanded 754 nodes. The reduction was .13% and both had an optimal solution cost of 38.855. For bear->duck my heuristic h_0 expanded 720 nodes while the ai's proposed heuristic expanded 714 nodes. The reduction was .83% and both had an optimal solution cost of 38.24.

The evidence shows that the proposed heuristic had slightly better results than h_0 . For why->ask it expanded 1 less node and it expanded 6 less nodes for bear-duck. Both heuristics still found the same exact optimal solution cost in the end. Since both heuristics found the same optimal cost, and one did only slightly better than the other, I wouldn't say that this is strong evidence to fully conclude that the proposed heuristic is stronger than h_0 .

Experimental tests can reveal counterexamples, but passing a collection of tests does not by itself prove admissibility.

The AI system is not expected to be correct. Your grade is based on the quality of your verification, correction, and justification.

7. Collaboration, sources, and generative AI policy

You may discuss the assignment's concepts and possible approaches with classmates. You remain responsible for your final implementation, analysis, and understanding.

You may use:

- course materials and supplied code;
- Python and library documentation;
- other external explanatory resources; and
- generative AI systems.

6

Question 4 requires the use of a generative AI system.

If you use generative AI elsewhere in the assignment, briefly disclose:

1. the system used;
2. what you used it for; and
3. how you verified the resulting code or reasoning.

Complete prompt histories are not required except for the exact prompt and relevant output requested in Question 4.

Clearly cite any external code, text, or substantial solution idea that you copy or adapt. Do not submit another person's work or unattributed external material as your own. Mention if you have discussed with anyone. You are responsible for the correctness of everything you submit and should be able to explain how your implementation and written answers work.

Standard UMBC academic-integrity requirements continue to apply.

8. Submission

Submit:

1. dc.py, containing your completed DC implementation; and
2. pa01.pdf, containing your answers to the written-analysis questions.

Ensure that:

- your program runs with the supplied repository files;
- your name and section appear in pa01.pdf;
- all required experiments and tables are complete;

- any external resources are cited; and
- any generative AI use is disclosed as described above.

We may evaluate dc.py on additional valid DOGCAT problems not included in the visible tests.

Point summary

Component Points

DOGCAT implementation	60
Question 1: Heuristics and admissibility	15
Question 2: UCS and A* comparison	10
Question 3: Reversibility and costs	5
Question 4: LLM heuristic audit	10
Total	100